Enhancement Two: Algorithms and Data Structure

Artifact Description

The artifact is an interactive three-dimensional desktop scene originally created in CS 330: Computational Graphics and Visualization. I developed the original version as a C++ and OpenGL course project to demonstrate geometric modeling, transformations, textures, materials, lighting, and camera navigation. The scene contains a table, cup, candle, succulent plant, and laptop assembled from reusable primitive meshes. During the software engineering enhancement for CS 499, I improved the organization, reliability, resource handling, lighting, documentation, and project configuration. For Milestone Three, I used that improved version as the baseline and added a logical scene-object registry and object-picking system. The artifact now does more than display a static composition: a user can aim the camera at an object and click to identify and visually highlight the closest object intersected by a world-space ray.

The enhanced files include SceneObjectRegistry.h and SceneObjectRegistry.cpp, changes to MainCode.cpp, SceneManager.h and SceneManager.cpp, read-only camera matrix accessors in ViewManager, and a fragment-shader selection tint. A separate assertion-based test program verifies registration, lookup, bounding-box transformation, ray intersection, miss handling, and nearest-hit selection. The submission package contains both the original baseline and the enhanced artifact so that the growth is directly visible.

Justification for Inclusion

I selected this artifact because it provides a strong visual demonstration of algorithmic reasoning. The original project already had an understandable scene and camera, but its meshes were rendered through hard-coded statements and were not represented as searchable logical objects. That limitation created a meaningful enhancement opportunity: I could connect user

input, linear algebra, geometric intersection testing, data-structure selection, defensive validation, and visible feedback in one cohesive feature. The result shows not only that I can implement an algorithm, but also that I can place it into an existing application without discarding working rendering behavior.

Data Structure Design

The SceneObjectRegistry uses a hybrid design. A std::vector<SceneObject> owns the objects and maintains stable traversal order. A std::unordered_map<std::string, std::size_t> maps each stable object ID to its vector index. This avoids maintaining duplicate object copies while providing average O(1) direct lookup for rendering and selection-state checks. In comparison, using only a vector would make every ID lookup O(n), while using only an unordered map would not communicate intentional scene order as clearly. The registry rejects empty or duplicate identifiers and invalid bounds, which protects its internal invariant that every ID maps to exactly one valid vector element.

Ray Casting and Intersection

When the user presses the left mouse button, MainCode.cpp builds a center-screen picking ray. The algorithm uses the inverse projection matrix to move from clip space to eye space and the inverse view matrix to move from eye space to world space. The resulting direction is normalized and paired with the camera position. Center-screen selection is appropriate because the existing application captures the cursor for camera movement; the user aims the center of the view rather than switching between camera and pointer modes.

Each logical object stores local bounds and a model matrix. TransformBounds evaluates all eight corners of the local box and constructs a conservative world-space axis-aligned bounding box. IntersectRayAABB then uses the slab method. For each axis, it calculates the ray's entry and exit

distances, swaps them when the direction is negative, and progressively narrows the valid interval. A ray misses when the interval becomes empty. Nearly parallel axes receive a separate containment check, zero-length and non-finite rays are rejected, and intersections behind the camera are ignored. SelectNearest examines the scene objects in O(n) time and retains the smallest nonnegative distance, which ensures that the visually closest object is selected when multiple bounds overlap along the ray.

The O(n) scan is a deliberate trade-off. The scene contains only five logical objects, so a spatial tree would add construction, update, and maintenance complexity without a measurable user benefit. A bounding volume hierarchy or octree would become appropriate if the scene expanded to hundreds or thousands of objects. For the current scope, the linear pass is predictable, easy to test, and sufficiently efficient. The fragment shader provides immediate amber highlighting while retaining the selected object's texture, material, lighting, and transparency, making the algorithm's result observable to both users and reviewers.

Course Outcome Progress

This enhancement most directly demonstrates Course Outcome Three: designing and evaluating computing solutions using algorithmic principles and appropriate computer science practices while managing design trade-offs. I selected structures based on access patterns, analyzed lookup and traversal complexity, used matrix unprojection and the slab intersection algorithm, and deliberately chose a linear nearest-hit search over an unnecessarily complex spatial index. The code and test cases make those design decisions visible and repeatable rather than presenting selection as unexplained behavior.

The work also advances Course Outcome Four by applying C++, GLM, OpenGL, and GLSL to deliver a useful interactive capability within an existing graphics application. It

contributes to Course Outcome Five through defensive validation: duplicate and empty IDs are rejected, invalid bounds do not enter the registry, invalid rays fail safely, near-zero direction components avoid unstable division, a miss clears the previous selection, and read-only ViewManager getters limit unnecessary mutation of camera state. These choices are examples of a security mindset because the program verifies assumptions at trust boundaries instead of allowing malformed state to propagate.

Course Outcome Two is supported through the professional narrative, explanatory code comments, test table, console feedback, and visible selection cue. These formats communicate the same feature to technical and nontechnical audiences at different levels of detail. Course Outcome One is supported more indirectly. Stable object IDs, separated algorithm code, documented tests, and an explicit project structure make the enhancement easier for another developer or reviewer to discuss and extend. My overall outcome-coverage plan remains unchanged: this milestone provides the strongest evidence for Outcome Three, while the final portfolio, database enhancement, code-review video, and professional self-assessment will provide broader and more explicit evidence for collaboration, audience adaptation, database practices, and security.

Reflection on the Enhancement Process

The most important lesson from this enhancement was that object selection is not one isolated formula. It is a data-flow problem that crosses input handling, camera mathematics, world transforms, search, state management, rendering, and testing. The ray has to be represented in the same coordinate system as the bounds, and the selected logical object has to correspond to several meshes that together form the visible cup, plant, candle, or laptop.

Separating the logical object from its individual draw calls allowed one selection state to highlight every mesh in the group.

One challenge was selecting useful bounds for composite objects. The laptop, for example, contains a flat base and tilted screen, while the plant contains a pot and six leaves. Exact triangle-level tests would be more precise but would require substantially more computation and coupling to mesh data. I chose logical bounding boxes that enclose each complete object. This produces fast, understandable selection with a reasonable precision trade-off. Transforming all eight corners also keeps the algorithm valid when model matrices introduce rotation or nonuniform scale.

A second challenge was preserving the existing captured-mouse camera experience. A conventional cursor-position picking design would conflict with the callback that uses mouse movement to rotate the camera. I resolved this by using a center-screen ray and edge-triggered left-click input. Edge detection prevents one physical click from executing a selection every frame. This approach fits the existing interaction model and still demonstrates the complete clip-to-world ray algorithm.

Testing the geometry outside the graphics runtime improved my confidence in the design. The standalone test executable does not need a display or OpenGL context, so failures can be isolated from shader, driver, and texture issues. Eight test cases passed, including unique registration, duplicate rejection, known and unknown lookup, nearest-hit choice, miss behavior, transformed bounds, parallel-ray rejection, and invalid or backward ray handling. The complete Windows OpenGL program still requires Visual Studio and the supplied platform libraries for runtime verification; the Visual Studio project includes the new source file and retains the post-

build step that copies glew32.dll beside the executable. This distinction taught me to separate algorithm verification from platform integration verification and to report both honestly.

Overall, the enhancement changed the artifact from a scene that only renders objects into an application that can reason about them. I strengthened my understanding of vectors and hash maps, algorithm complexity, coordinate-space transformations, defensive numerical programming, and testable architecture. The feature also creates a foundation for future capabilities such as object manipulation, saved selections, metadata panels, or a spatial acceleration structure if the scene grows.

Verification Summary

Verification Area	**Result**	**Evidence**
Registry behavior	Passed	Unique insertion, duplicate defense, and hash lookup assertions
Geometric algorithm	Passed	Hit, miss, nearest, transformed bounds, parallel, and invalid-ray assertions
Project integration	Configured	New source/header files included in the Visual Studio project
Windows rendering	Final local check	Build and run in Visual Studio; aim and left-click to verify highlight